TRAINING AND SCALING LANGUAGE MODELS: FROM FIRST PRINCIPLES TO EFFICIENT SERVING

Module 1: Transformer from First Principles

Building a Mental Model of a Decoder-Only LM

BRAIN team

IMT Atlantique · January 2027

IMT Atlantique
Bretagne-Pays de la Loire
École Mines-Télécom

BRAIN

Lecture map

1. Next-token prediction
2. Attention from first principles
3. The Transformer block
4. From GPT-2 to modern LLMs & Lab 1

Guiding question

What must happen between a token ID and a distribution over the next token—and how do we turn that into verified code?

What should be clear by the end?

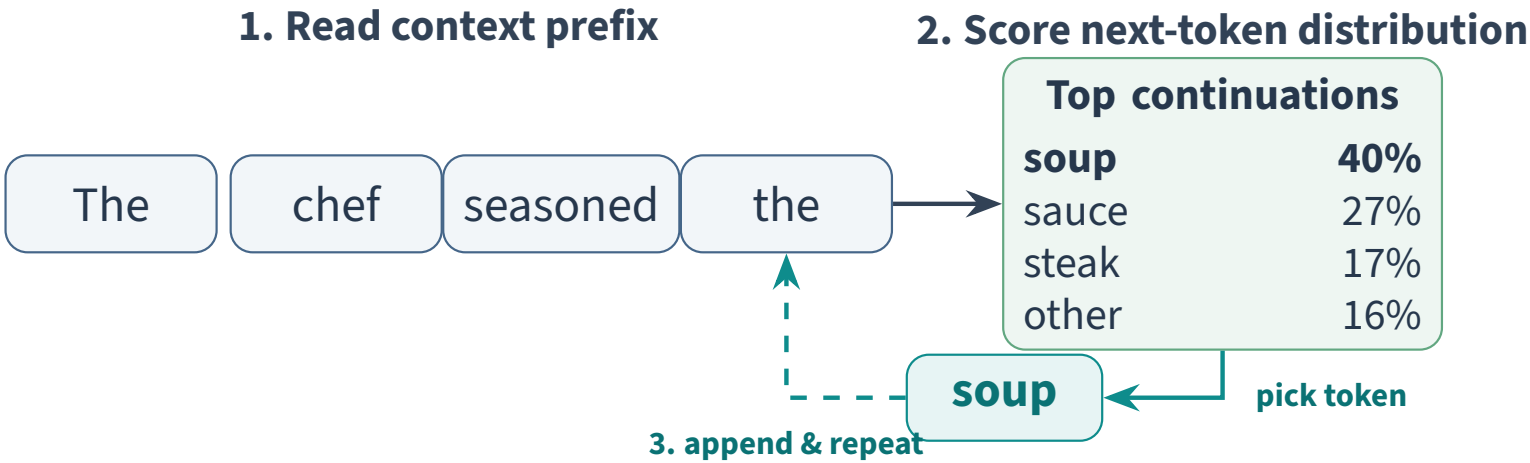
1. Formulate next-token prediction and explain why a Transformer trains all positions in parallel.
2. Derive causal multi-head self-attention and positional encoding from first principles.
3. Trace tensor shapes, parameters, and gradients through a pre-norm Transformer block (MHA, MLP, LayerNorm, LM head).
4. Contrast our GPT-2 baseline with modern LLMs (LLaMA/Mistral) and verify our implementation against official weights.

PART 1

Next-token prediction

1. Next-token prediction
2. Attention from first principles
3. The Transformer block
4. From GPT-2 to modern LLMs & Lab 1

The task: sequence in, next-token distribution out



At every step, use the prefix to score the vocabulary $\mathcal{V}$, append one token, and repeat.

▷ **Live demo: Explore next-token prediction (next-token.html)**

Formalizing next-token prediction (NTP)

Autoregressive factorization

By the chain rule, the joint probability of $X = (x_1, \ldots, x_T)$ factorizes exactly into conditional steps:

$$P(X) = \prod_{t=1}^{T} P(x_t \mid x_1, \ldots, x_{t-1})$$

Modeling a text sequence is equivalent to predicting the next token from every prefix $x_{<t}$.

Self-supervised training loss

Natural text is self-labeled: each token x_t is the ground-truth target for prefix $x_{<t}$. We minimize cross-entropy:

$$\mathcal{L}(\theta) = -\frac{1}{T} \sum_{t=1}^{T} \log P_\theta(x_t \mid x_{<t})$$

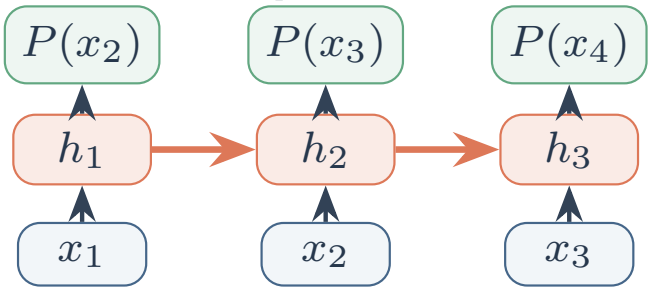
One sequence of length T provides T supervised prediction targets at once.

Source: Bengio et al. (2003); Radford et al. (2018, 2019)

Same task, two ways to process the sequence

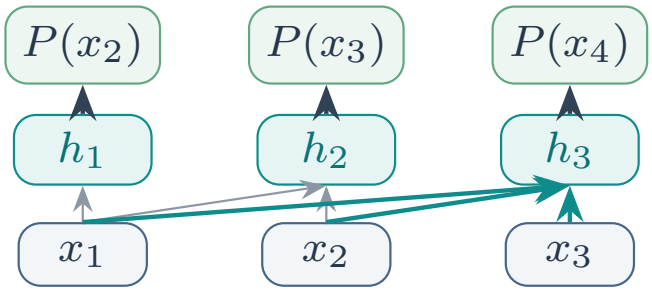
At every position t, predict $P(x_{t+1} \mid x_{\leq t})$ from the prefix so far.

Recurrent model (RNN): inherently sequential



$h_t = f(h_{t-1}, x_t)$: step t waits on step $t-1$; gradients vanish/explode over long chains.

Causal Transformer: inherently parallel



No $h_{t-1} \to h_t$ wait: every position gathers $x_{\leq t}$ directly via attention in one pass.

Hardware alignment: parallelism over operation count

Transformers restructure sequence learning into dense matrix multiplications (GEMMs) so all T training positions execute concurrently on GPUs.

Source: Elman (1990); Hochreiter & Schmidhuber (1997); Pascanu et al. (2013); Vaswani et al. (2017)

The whole model, at a glance



How do tokens interact inside a Transformer block?

Without a recurrent state h_{t-1}, we need a mechanism that lets each token selectively pull information from earlier tokens in parallel: **self-attention**.

Source: Vaswani et al. (2017); Radford et al. (2019)

PART 2

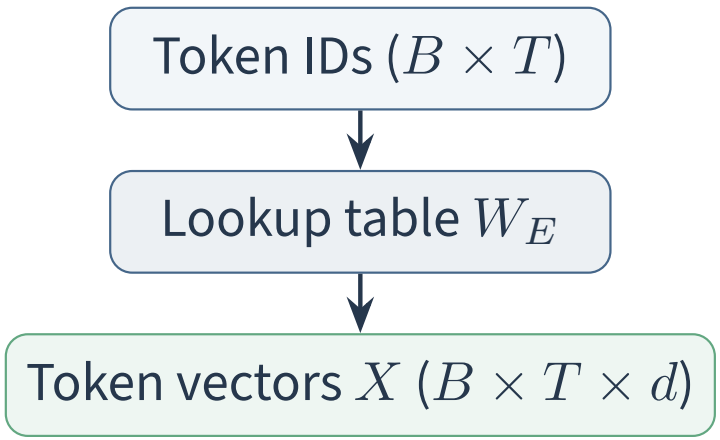
Attention from first principles

1. Next-token prediction
2. Attention from first principles
3. The Transformer block
4. From GPT-2 to modern LLMs & Lab 1

Step 1: Tokens become vectors in a shared space

- Input token IDs are integers in $\{0, \ldots, V-1\}$ with shape $B \times T$.
- An embedding lookup table $W_E \in \mathbb{R}^{V \times d}$ maps each token ID to a vector of width d:

$$X = W_E[\text{ids}] \quad \in \mathbb{R}^{B \times T \times d}$$

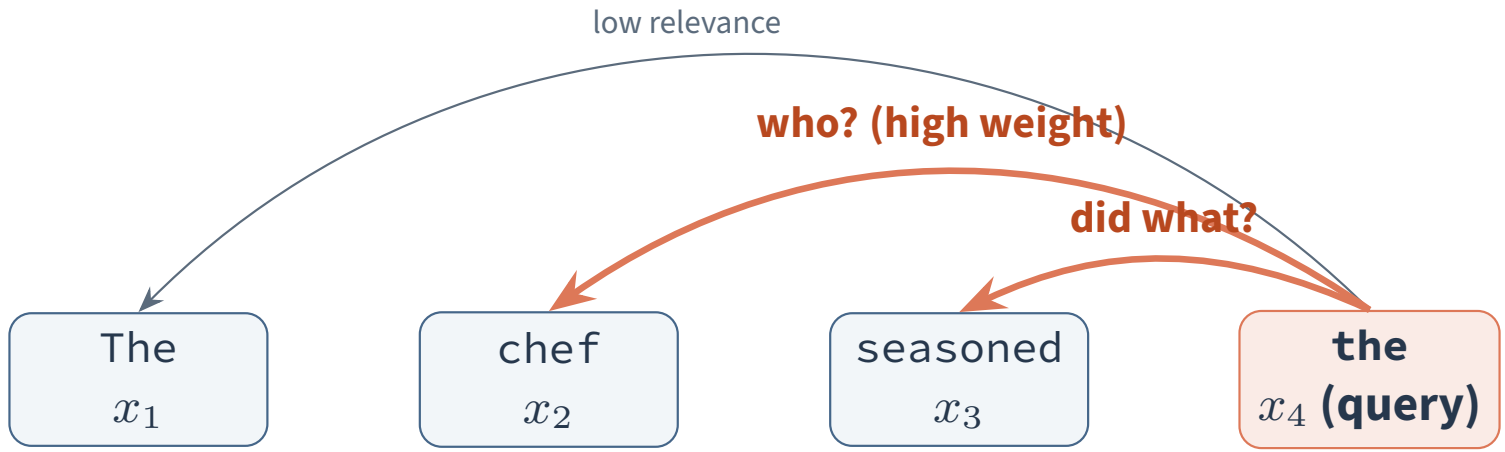


Isolated representations

At this stage, each row $x_t \in \mathbb{R}^d$ only knows its own token identity—it knows nothing about surrounding tokens.

Step 2: Relating tokens across the sequence

To predict what follows **“the”**, its representation must pull context from related earlier tokens:



The goal of self-attention

For every token i, compute a relevance weight $\alpha_{i,j} \geq 0$ ($\sum_{j \leq i} \alpha_{i,j} = 1$) for each allowed token $j \leq i$, and form a weighted mixture of their information.

Source: Bahdanau et al. (2014); Vaswani et al. (2017)

Step 3: Queries, Keys, and Values

Each token vector x_t is projected into three distinct vectors via learned matrices W_Q, W_K, W_V:

Query ($q_t = x_t W_Q$)
What context is this token looking for?

Key ($k_t = x_t W_K$)
What features does each token offer?

Value ($v_t = x_t W_V$)
What content moves when selected?

- **Matching**: Compare token i's query q_i with token j's key k_j via dot product $s_{i,j} = q_i^\top k_j$.
- **Retrieving**: Normalize scores with softmax into weights $\alpha_{i,j}$ and mix values: $\text{output}_i = \sum_{j \le i} \alpha_{i,j} v_j$.

Why three projections instead of using x_t directly?

Asymmetry: what token i searches for (q_i) differs from the features it offers (k_i) and the content it transmits (v_i).

Step 4: Computing attention for all tokens in parallel

Stacking queries, keys, and values into matrices $Q, K, V \in \mathbb{R}^{T \times d_h}$ computes all pairwise relationships in one formula:

$$\mathrm{Attn}(Q,K,V) = \underbrace{\mathrm{softmax}\Big(\overbrace{QK^\top}^{\text{1. similarity scores}} + \overbrace{M}^{\text{2. mask}}\Big)}_{\text{3. attention weights } A \in \mathbb{R}^{T \times T}} \underbrace{V}_{\text{4. value mix}}$$

1. Score pairs
$S = QK^\top$
all (i, j) dot products

→

2. Causal mask
$S + M$
hide future $j > i$

→

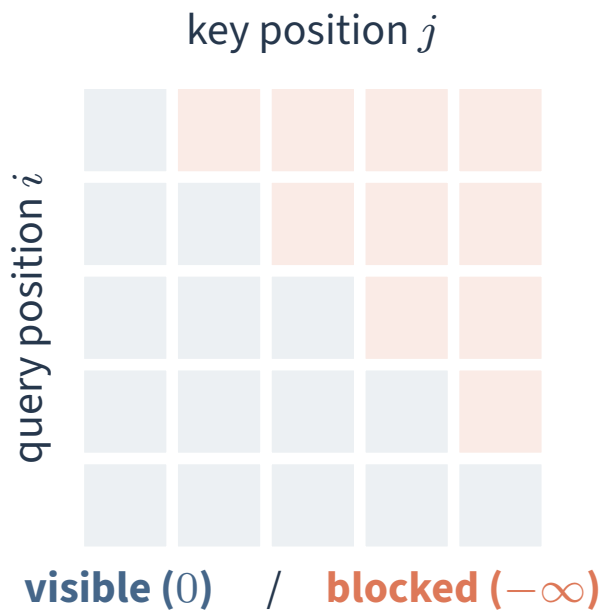
3. Softmax
$A = \mathrm{softmax}(\cdot)$
row weights sum to 1

→

4. Mix values
$O = AV$
contextualized states

Source: Vaswani et al. (2017); Rush (2018, The Annotated Transformer) — in code, divide scores by $\sqrt{d_h}$ to keep logit variance ≈ 1

Causality is a constraint on logits



- Position i may attend only to keys $j \leq i$.
- Future scores ($j > i$) receive $-\infty$ **before** softmax so $\exp(-\infty) = 0$.
- Zeroing probabilities after softmax breaks row normalization.

PyTorch implementation tip

Register a lower-triangular mask via `register_buffer` and apply `scores.masked_fill(mask == 0, -inf)`.

Checkpoint: one query row

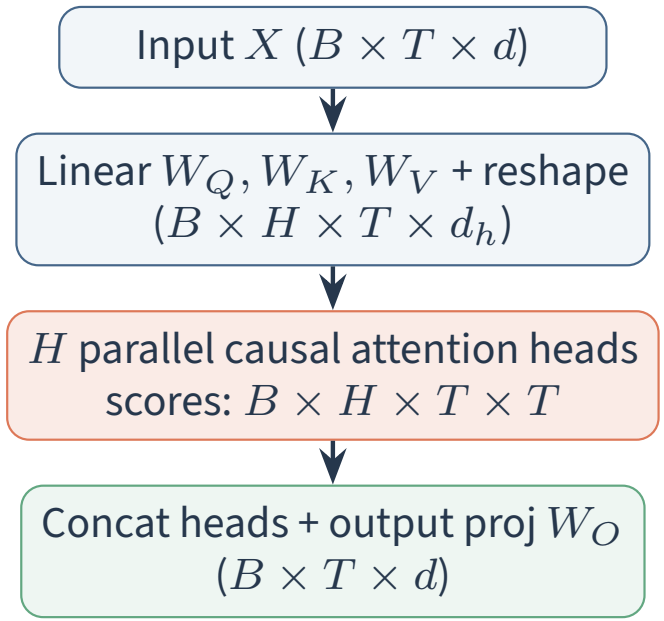
Think, then compare

For sequence length four, write the four logits seen by the query at position two (0-indexed). Which entries survive the mask, and across which dimension must softmax normalize?

Multi-head attention: multiple parallel retrieval subspaces

One token needs multiple types of context at once (syntax, coreference, local phrases). We split width d into H heads of size $d_h = d/H$:

1. **Project**: $Q, K, V \in \mathbb{R}^{B\times T\times d}$ via $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$.
2. **Split heads**: `view(B, T, H, d_h).transpose(1, 2)` $\rightarrow B \times H \times T \times d_h$.
3. **Attend per head**: $S = QK^\top/\sqrt{d_h} \in \mathbb{R}^{B\times H\times T\times T}$, mix $O_h = AV_h$.
4. **Merge & project**: `transpose(1, 2).view(B, T, d)` then multiply by $W_O \in \mathbb{R}^{d\times d}$.



Transpose trap

Always `transpose(1, 2)` between T and H so each head sees a full $T \times d_h$ sequence!

Source: Vaswani et al. (2017)

Wait—attention alone is blind to token order!

Permutation problem

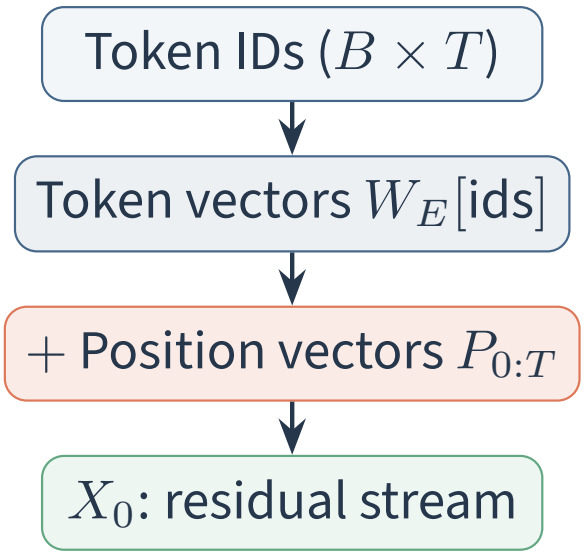
Dot products $q_i^\top k_j$ depend only on token vectors, not their indices. Without positions, attention treats the prefix like an unordered **bag of words**.

Solution: Positional Encoding (PE)

Inject position information so identical words at different positions have distinct vectors:

$$X_0 = W_E[\text{ids}] + P_{0:T} \quad \in \mathbb{R}^{B\times T\times d}$$

GPT-2: learned $W_{pe} \in \mathbb{R}^{T_{\max}\times d}$. **Modern LLMs**: RoPE rotates Q, K by relative distance.



Source: Vaswani et al. (2017); Radford et al. (2019); Su et al. (2021, RoPE)

PART 3

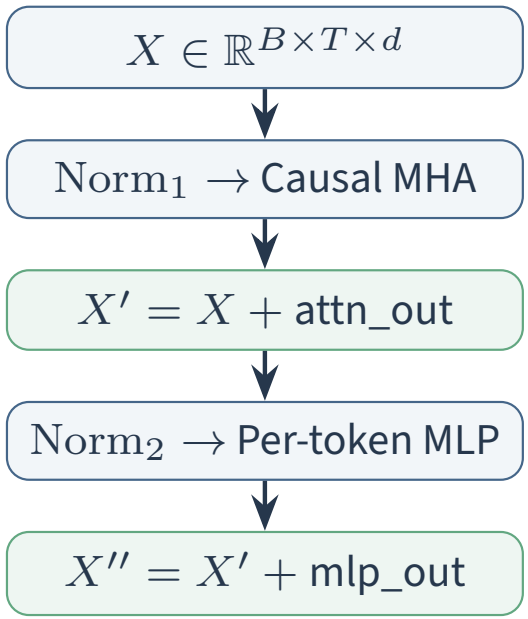
The Transformer block

1. Next-token prediction
2. Attention from first principles
3. The Transformer block
4. From GPT-2 to modern LLMs & Lab 1

A pre-norm Transformer block has two residual updates

$$X' = X + \mathrm{MHA}(\mathrm{Norm}_1(X))$$
$$X'' = X' + \mathrm{MLP}(\mathrm{Norm}_2(X'))$$

- **Self-attention** routes information **across positions** (sequence axis T).
- **The MLP** transforms features **within each position** independently (channel axis d).
- **Residual highway** ($X + \dots$) lets gradients flow unimpeded to early layers.



Source: He et al. (2016, ResNet); Radford et al. (2019); Xiong et al. (2020)

Inside the MLP and the $1/3$ vs. $2/3$ parameter rule

The MLP applies a two-layer feed-forward network to each token vector $x_t \in \mathbb{R}^d$ independently:

$$\mathrm{MLP}(x_t) = \mathrm{GELU}(x_t W_1 + b_1) W_2 + b_2$$

- **Expand**: $W_1 \in \mathbb{R}^{d \times 4d}$ projects $d \to 4d$.
- **Activate**: elementwise GELU non-linearity.
- **Contract**: $W_2 \in \mathbb{R}^{4d \times d}$ projects $4d \to d$.

Parameter budget per block

- **MHA** (W_Q, W_K, W_V, W_O): $\mathbf{4d^2}$ **(1/3)**
- **MLP** (W_1, W_2): $\mathbf{8d^2}$ **(2/3)**
- **Total per block**: $\mathbf{12d^2}$ parameters

Takeaway for scaling

Two-thirds of block parameters and matrix-multiply FLOPs live in the MLP!

Source: Vaswani et al. (2017); Hendrycks & Gimpel (2016, GELU)

Normalization: stabilizing activations and gradients

LayerNorm (per token across d)

For each token vector $x \in \mathbb{R}^d$:

$$\mathrm{LN}(x) = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta$$

Centers to zero mean (μ), scales to unit variance (σ^2), then applies learned $\gamma, \beta \in \mathbb{R}^d$.

Pre-Norm vs. Post-Norm

- **Pre-Norm (GPT-2 / modern LLMs)**: $X + F(\mathrm{Norm}(X))$. Identity highway bypasses norm; stable gradients at initialization.
- **Post-Norm (2017 Transformer)**: $\mathrm{Norm}(X + F(X))$. Requires careful warmup; different model!

Modern speedup: RMSNorm drops mean-centering (μ) and β, normalizing only by $\sqrt{\frac{1}{d}\sum_i x_i^2 + \epsilon}$.

Source: Ba et al. (2016, LayerNorm); Zhang & Sennrich (2019, RMSNorm); Xiong et al. (2020)

Closing the loop: final norm, LM head, and target shift

After N Transformer blocks, normalize X_N and project to vocabulary logits:

$$Z = \mathrm{Norm}_f(X_N) W_{\text{vocab}}^{\top} \qquad \in \mathbb{R}^{B \times T \times V}$$

$$\mathcal{L} = -\frac{1}{BT} \sum_{b,t} \log \mathrm{softmax}(Z_{b,t})_{y_{b,t}}$$

- **Weight tying (GPT-2)**: shares $W_{\text{vocab}} = W_E \in \mathbb{R}^{V \times d}$, saving $V \times d$ parameters (≈ 38.6M in GPT-2 Small).

One stream, two views

```
x = tokens[:, :-1]
y = tokens[:, 1:]
```

Common bug

Forgetting the $+1$ shift trains the model to copy the current token ($x_t \to x_t$): loss drops to zero, generation fails completely.

Source: Press & Wolf (2017, Weight Tying); Radford et al. (2019)

Interactive walkthrough: visual explorer of the Transformer

Session cue — trace a live forward pass:

1. Follow **token + position embeddings** into the residual stream.
2. Zoom into the **Transformer block**: inspect Q, K, V projections, causal attention weights, and head concatenation.
3. Trace the **MLP expansion/contraction** and final **LM head** to next-token logits.

Visual Explorer Demo

Open the interactive canvas and hover over each component to inspect tensor shapes and data flow:

▷ **Open decoder-transformer.html**

See also: Transformer Explainer (GPT-2 live)

PART 4

From GPT-2 to modern LLMs & Lab 1

1. Next-token prediction
2. Attention from first principles
3. The Transformer block
4. From GPT-2 to modern LLMs & Lab 1

Three Transformer families—and why Decoder-Only won

Encoder-only

BERT (Devlin et al., 2018)

Bidirectional attention for classification and search. No autoregressive generation.

Encoder–decoder

Original / T5 (Raffel, 2020)

Bidirectional source encoder + causal target decoder via cross-attention.

Decoder-only

GPT / LLaMA (Radford, 2019)

Single causal stream unifies all tasks as prompt completion + single KV cache.

Why decoder-only dominates modern foundation models

Any task can be framed as prompt continuation, concentrating all parameters and pretraining compute into one unified causal stack with a single KV cache.

Source: Vaswani et al. (2017); Devlin et al. (2018); Radford et al. (2019); Brown et al. (2020)

What changed since GPT-2? The modern LLM stack

You will implement **GPT-2** in Lab 1. Modern LLMs keep the same macro-architecture while upgrading five components:

Component	GPT-2 (2019)	Modern LLM (2023+)	Primary rationale
Positions	Learned W_{pe}	**RoPE** (Su et al., 2021)	Relative distance, length extrapolation
Normalization	Pre-**LayerNorm**	Pre-**RMSNorm** (Zhang, 2019)	Drops mean-centering; 10–50% faster
Attention	**MHA** ($H_Q = H_{KV}$)	**GQA** (Ainslie et al., 2023)	$4\times$–$8\times$ smaller KV cache in serving
MLP activation	**GELU** ($d \to 4d \to d$)	**SwiGLU** (Shazeer, 2020)	Gated linear unit improves capacity
Biases & tying	Biases; tied W_E	**Bias-free**; untied W_{vocab}	Faster GEMMs; better stability at scale

Source: Touvron et al. (2023, LLaMA); Jiang et al. (2023, Mistral); Raschka (LLM Architecture Gallery)

A shape ledger catches semantic bugs

Tensor	Shape	Role in `nanolm/model.py`
`idx`	$B \times T$	input token IDs ($T \leq$ block_size)
`x`	$B \times T \times d$	residual stream (token + position)
`q, k, v`	$B \times H \times T \times d_h$	per-head projections ($d_h = d/H$)
`att`	$B \times H \times T \times T$	masked attention weights (rows sum to 1)
`mlp_hidden`	$B \times T \times 4d$	expanded per-token MLP features
`logits`	$B \times T \times V$	next-token vocabulary scores

Review question

Over which axis of `att` ($B \times H \times T \times T$) does `softmax` normalize, and why must masking happen before it?

Lab 1: what you have the tools to build and verify now

Implement in `nanolm/model.py`

1. **`CausalSelfAttention`**: W_Q, W_K, W_V, head split, $QK^\top/\sqrt{d_h}$, causal buffer mask, W_O.
2. **`MLP` & `TransformerBlock`**: $d \to 4d \to \mathrm{GELU} \to d$ and pre-LN residual updates.
3. **`MiniGPT`**: $W_E + W_{pe}$, N blocks, final LN, tied `lm_head`, cross-entropy loss.

Three verification gates

1. **Shape tests**: all intermediates match the ledger.
2. **Causality tests**: mutating a future token leaves earlier logits identical.
3. **Official GPT-2 parity**: load 124M weights (`01_gpt2_parity.py`) and match HF logits on `"The capital of France is"` → `" Paris"`.

`uv run pytest tests/test_module_01_model.py -q` → `uv run --extra gpt2 python scripts/01_gpt2_parity.py`

Checkpoint: debugging order

Think, then compare

Your model passes all shape tests, but `01_gpt2_parity.py` produces wrong logits and the causality test fails. Which lines of `CausalSelfAttention.forward` do you inspect first?

The durable ideas

1. Next-token prediction factorizes sequence probability; removing the $h_{t-1} \to h_t$ dependency makes training inherently parallel across positions on GPUs.
2. Causal multi-head attention routes information across tokens ($1/3$ of params), while positional encodings supply token order.
3. A pre-norm Transformer block pairs cross-token attention with a per-token $4d$ MLP ($2/3$ of params) along a clean residual highway.
4. Shape invariants, causality perturbation tests, and exact GPT-2 checkpoint parity verify the model before we train it in Module 2.